

SEO-Keyword → ContentBrief Pipeline

für zvoove

500 Keywords · 13 thematische Cluster

Bewerbung als Revenue AI Architect

Agenda

— hier später Outline einfügen —

1. Aufgabe
2. Anforderungen & mein Ansatz
3. Ergebnis
4. Architektur
5. Cluster-Schritt im Detail
6. Briefs & Reporting
7. Drei Empfehlungen für zvoove
8. Limits & nächste Schritte

Was ist die Aufgabe?

Automatisiere den Prozess:

Quelle → Keywords → Cluster → Content Brief → Reporting

Beispiel:

zvoove.de/wissen/blog → 500 Keywords → 13 Cluster → 13 Briefs → Dashboard

Warum?

Im Bereich Zeitarbeit und Personaldienstleistung **organischen Traffic gewinnen, der echte Kaufinteressenten bringt.**

Leitkriterien

1 · Flexible Integration in bestehende Systeme

Daten können per API oder CSV angebunden werden. Ergebnisse werden als JSON oder CSV bereitgestellt und lassen sich technisch problemlos in Google Sheets, Airtable, Notion oder ein CMS erweitern.

2 · Provider-unabhängig, kein Vendor Lock-in

Der LLM-Call ist abstrahiert und kann über verschiedene Anbieter laufen: heute Anthropic, morgen OpenAI oder ein lokales Modell — ohne Umbau der Pipeline.

3 · Wartbar, validiert und übergabefähig

Die Pipeline ist versioniert, reproduzierbar und durch 42 Tests abgesichert. Die Cluster-Qualität wird zusätzlich über etablierte Metriken wie Silhouette und ARI validiert.

4 · Modularer, testbarer Aufbau

Der Prozess besteht aus sechs klar getrennten Schritten. Jeder Schritt ist einzeln testbar, nachvollziehbar und bei Bedarf austauschbar.

Ergebnis in zwei Zahlen

500

Keywords

13

Cluster

240k

Suchen / Monat

Validiert mit Silhouette 0,65 und ARI 0,81 gegen ein zweites Verfahren.

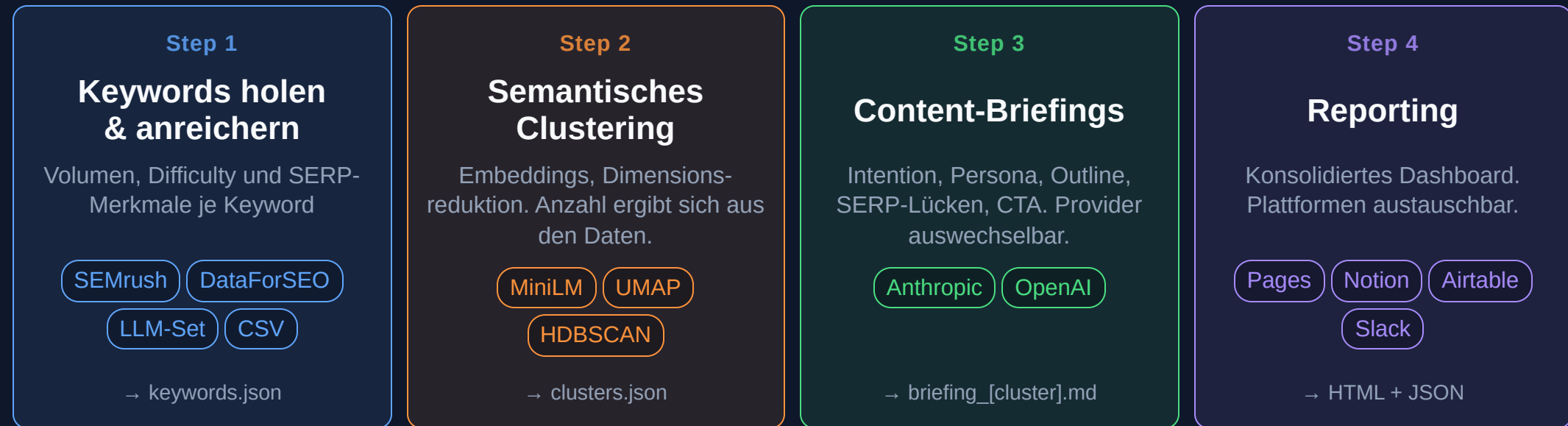
Drei interessante Cluster

Cluster	SV/Monat	Kommerziell	Hebel
HR- & Dokumentenverwaltungssoftware	45.000	89 %	Bottom-of-Funnel
Zvoove Plattform & Preise	23.000	97 %	Brand Defense
Digitalisierung Personaldienstleistung	24.000	35 %	Top-of-Funnel

Vollständige Tabelle aller 13 Cluster im Dashboard.

Automatisierung

GitHub Actions · Auslöser per Cron-Schedule oder manueller Trigger



Infrastruktur · unsichtbar aber überall

Secrets

Run-Logs

Retry-Logik

Git-Versionierung

Discover: ehrlicher Stand

Aktuell: Stub. Liest ein kuratiertes Keyword-Set aus CSV.

Geplant: Live-Scraping vom Blog plus Claude-basierte Keyword-Expansion.

Warum nicht jetzt?

Web-Scraping ist konzeptionell der schwierigste Schritt: Anti-Bot, JavaScript-Rendering, Pagination.
Lieber **vier Schritte richtig fertig** als sechs halb fertig.

Trade-off transparent in den Architecture Decision Records dokumentiert.

Cluster-Schritt: was ist ein Embedding?

Ein **Embedding** ist eine Zahlenfolge, die die Bedeutung eines Texts beschreibt.

Beispiel:

Lohnabrechnung Software $\approx$ Payroll Tool

Andere Wörter, gleiche Bedeutung, **ähnliche Zahlen**.

Modell: `paraphrase-multilingual-MiniLM-L12-v2` · mehrsprachig · 120 MB · läuft ohne GPU

Cluster-Schritt: warum HDBSCAN?

k-means

- Clusteranzahl muss vorgegeben werden
- Kein Outlier-Konzept
- Setzt sphärische Cluster voraus

HDBSCAN ✓

- Findet die Anzahl selbst
- Markiert Ausreißer als Rauschen
- Variable Cluster-Dichte

Beispiel: `fachkräftemangel deutschland` — gehört semantisch zu nichts. HDBSCAN sagt das. k-means würde es zwanghaft zuordnen.

Woran erkennt man gutes Clustering?

Kein Goldstandard ohne Labels —
deshalb validieren.

Nicht eine einzelne Kennzahl entscheidet — sondern ein konsistentes Gesamtbild.

Trennschärfe

Innerhalb ähnlich, zwischen Clustern
verschieden

1

Silhouette = 0,65

solide Trennung
(> 0,5 gilt als gut)

Stabilität

Unabhängiges Verfahren findet
ähnliche Struktur

2

ARI = 0,81

hohe Übereinstimmung
mit Ward(k=10)

Plausibilität

Parameter systematisch und
Segmente interpretierbar

3

Grid Search

reproduzierbar,
nichts geraten

Fazit Die Cluster sind nicht zufällig, sondern belastbar und interpretierbar.

Lesart: gutes Clustering = getrennt + stabil + reproduzierbar + fachlich sinnvoll
Damit ist die Segmentierung stark genug für die nächste Ebene: Profilierung und fachliche Interpretation der Cluster.

Briefs mit Claude + Prompt Caching

Pro Cluster ein Markdown-Brief mit:

Hauptkeyword, Suchintention, Zielgruppe, Outline (H1–H3), 3 Benchmark-URLs, CTA.

Prompt Caching: System-Prompt wird einmal gecached, 13× wiederverwendet

→ rund **90 % Token-Ersparnis**, < 1 USD pro Lauf.

Robust: Retry mit Backoff. Wenn ein Brief fehlschlägt, läuft die Pipeline weiter. Status-Bericht am Ende.

Reporting & Export

Report: eine HTML-Datei mit KPIs, Cluster-Tabelle, Charts, Karten-Link.
Bewusst kein Frontend-Framework. Verschickbar per Mail oder Slack.

Export: drei JSON-Dateien

Datei	Inhalt
<code>clusters.json</code>	eine Zeile pro Cluster
<code>keywords.json</code>	eine Zeile pro Keyword
<code>report.json</code>	alles zusammen

Optional: direkter Sync nach Airtable oder Google Sheets, per Schalter.

Empfehlung 1: HR-Software

45.000 Suchen / Monat

89 % kommerziell · KD ø 53 · 45 Keywords

Top-Keywords:

dokumentenmanagement software · bewerbermanagement software ·
mitarbeiterverwaltung software · hr software kmu

Was tun: Pillar-Pages zu Software-Kategorien, jeweils mit zvoove-Modul als Lösung.

Hypothese: 5 % CTR × 2 % MQL-Rate ≈ **45 MQLs / Monat.**

Empfehlung 2: zvoove-Marken-Keywords

23.000 Suchen / Monat

97 % kommerziell · KD ø 52 · 34 Keywords

Auffällig: KD 52 für Brand-Begriffe ist **ungewöhnlich hoch**.

→ Vergleichsseiten und Bewertungsportale belegen die SERP.

Was tun: zvoove-Erfahrungen-Hub unter `/produkte/`, der positive Bewertungen aggregiert.

Defense und Offense in einem.

Empfehlung 3: Digitalisierung

24.000 Suchen / Monat

35 % kommerziell · KD ø 36 · 37 Keywords · **Top-of-Funnel**

Top-Keywords:

digitalisierung zeitarbeit · künstliche intelligenz personaldienstleistung ·
digitale zeiterfassung

Was tun: Hub </wissen/digitalisierung-personaldienstleistung/>, der Awareness-Traffic in die kommerziellen Cluster überführt.

Wirkung: Pipeline-Influence über 6–12 Monate, nicht direkte Conversion.

Limits & nächste Schritte

Was fehlt

- Discover ist Stub
- Keine Datenbank, nur Dateien
- Keine GSC-Anbindung

Was als Nächstes

1. Discover live machen
2. Search Console anbinden
3. SQLite-Persistenz
4. CMS-Integration (Sanity)

Bei einer zweiten Iteration: Discover zuerst bauen, nicht zuletzt.

Was diese Pipeline zeigen soll

Architektur-Denken statt Skript-Denken

→ jeder Schritt einzeln ersetzbar

Pragmatismus statt Polish

→ Heuristik klar markiert, Live-Daten optional

Revenue-Lens auf alles

→ jede Empfehlung mit MQL-Hypothese

Danke.

Repo: `github.com/t1nak/seo-pipeline`

Live-Dashboard: `t1nak.github.io/seo-pipeline`

Fragen?